

Assignment II: SDD, BDD, and TDD, 3 methodologies in AI-Assisted Software Development

Student Information:

-Name: Wei-li Lin
-Student ID: 1123533
-Course: AI-Assisted Software Development
-Date: 2026-05-30

1. Introduction

In the current age of software development and computer science, artificial intelligence is gradually being involved in the software engineering process. With AI Assistance in Software Development, code generation, documentation, test cases, and other software artifacts can be done more swiftly and efficiently. As advancements in AI technology continue, they are gradually become a vital tool in modern software development workflows. One could even say that AI is slowly dominating every aspect of the tech industry. To organize development workflow and ensure efficiency and coherence, certain methods or approaches have been conceptualized. The three specific methods are SDD, BDD, and TDD.

There are cases where AI does not automatically understand the developer's exact intentions or project requirements. Without clear instructions plus well-defined objectives, AI-generated results can be inaccurate, incomplete, or inconsistent with the desired outcome. Therefore, developers must provide precise requirements, expected behaviors, and validation criteria to effectively guide AI-assisted development.

This is where Software Design Description, Behavior-Driven Development, and Test-Driven Development become helpful. SDD helps define the system's requirements and specifications, BDD describes how the system should behave from a user's point of view, and TDD establishes test cases that verify the system's functionality. Altogether, these approaches help create a structured framework that improves communication, reduces ambiguity, and helps ensure that AI-generated software artifacts align with the intended design and functionality.

2. Definition of SDD

SDD, or Specification-Driven Development, is the approach that treats specifications as the ultimate focus in all development aspects. This ranges from architecture design to testing & deployment. By getting requirements defined clearly at the beginning of a project, developers can ensure that the final system meets the intended functionality and business objectives. SDD also helps reduce vagueness and provides a common reference for both developers and AI-assisted development tools.

3. SDD: Student Grade Calculator

The Student Grade Calculator computes a student's final score using weighted assessment via components to determine the corresponding letter grade.

3.1 Goal

The Student Grade Calculator's main purpose is to determine a student's final score using weighted assessment via 4 components and determines the corresponding letter grade according to predefined rules for grading. The system shall output the final score rounded to one decimal place and display the appropriate letter grade.

3.2 Functional Requirements

FR-1: Calculate Final Score

The overall grade is determined by the weighted average formula below:

Final Score = (Assignment * 0.30) + (Midterm * 0.20) + (Finals * 0.30) + (Project * 0.20)

FR-2: Round Final Score

The system shall round the final score to one decimal place before displaying the result.

FR-3: Assign Letter Grade

The system shall assign a letter grade based on the rounded final score according to the grading rules.

FR-4: Validate Input

The system shall only accept numeric scores between 0 to 100 for all assessment components. \

3.3 Input

The following inputs are accepted by the system: |Component|Range|Weight| | ----- |-----|-----| | Assignment|0-100| 30% | |Midterm |0-100| 20% | |Final |0-100| 30% | |Project |0-100| 20% | All inputs must be numeric values within the range.

3.4 Output

The system shall display:

1. Final Score (rounded to one decimal)

2. Letter Grade Example Output:

Final Score: **84.2**

Letter Grade: **B**

3.5 Grade Rules

Grade Determination Rules |Final Score|Letter Grade| |-----|-----| |90 < Score < 100 | A | |80 < Score < 90 | B | |70 < Score < 80 | C | |60 < Score < 70 | D | |Score < 60 | F |

3.6 Acceptance Criteria

AC-1: Correct Score Calculation |Assignment|Midterm|Finals|Project|Final Score|Grade| | ----- |-----|-----|-----|-----| | 82 | 76 | 88 | 90 | 84.2 | B |

Given: that Assignment = 82, Midterms = 76, Final Exams, 88, and Project = 90.

When the system computes the final score,

then the result shall be **84.2**.

And the assigned grade will be **B**.

AC-2: Grade Boundary Verification **Given:** a final score of 90.

When the final grade is determined, then the assigned grade shall be **A**.

AC-3: Input Validation Given a project score of = 105.

When the user submits the scores,

Then the system shall **reject** the input.

And displays an error message informing that scores must be between 0 to 100.

4. Definition of BDD

The Behavior-Driven Development is an approach in software development that uses the user's perspective through natural language to describe system behavior. Scenarios belong to this category are usually written in the Given-When-Then format to specify conditions, actions, and expected results of a system.

5. BDD: Student Grade Calculator

1. Scenario 1: Student receives the B grade

Given the assignment is 84

And the Midterm score is 78

And the Final Score is 88

And the Project Score is 85

When the system calculates the final grade

Then the final score should be 84.0 And the Letter Grade should be B.

2. Scenario 2: Invalid project score entered

Given the assignment score is 75,

And the Midterm score is 80

And the Final Score is 82

And the Project Score is 125

When the system validates the input scores

Then the system should reject the input And an error message should indicate that scores must be between 0 and 100.

6. Definition of TDD

Test-Driven Development involves several test cases, usually 3 of them. Its process is basically writing test case/s that fail, then writing more code to the point that it's enough to pass the test, and lastly improving the code quality without drastic changes to its behavior. This process is usually described as "Red, Green, Refactor". This method helps developers detect defects early on and ensures that the software continuously satisfies its requirements.

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Test Case 1:

Input

- Assignment: 78
- Midterm: 82
- Finals: 85
- Project: 80

Expected Calculation Final Score = $(78 * 0.30) + (82 * 0.20) + (85 * 0.30) + (80 * 0.20)$

Final Score = $23.4 + 16.4 + 25.5 + 16.0$

Final Score = 81.3

Expected Outcome

- Final Score: 81.3
 - Letter Grade: B
-

Test Case 2:

Input

- Assignment: 92
- Midterm: 88
- Finals: 94
- Project: 90

Expected Calculation Final Score = $(92 * 0.30) + (88 * 0.20) + (94 * 0.30) + (90 * 0.20)$

Final Score = $27.6 + 17.6 + 28.2 + 18.0$

Final Score = 91.4

Expected Outcome

- Final Score: 91.4
 - Letter Grade: A
-

Scenario 2: Boundary Test Cases

Test Case 1:

Input

- Assignment: 90
- Midterm: 90
- Finals: 90
- Project: 90

Expected Calculation Final Score = $(90 * 0.30) + (90 * 0.20) + (90 * 0.30) + (90 * 0.20)$

Final Score = $27.0 + 18.0 + 27.0 + 18.0$

Final Score = 90.0

Expected Outcome

- Final Score: 90.0
 - Letter Grade: A
-

Test Case 2:

Input

- Assignment: 80
- Midterm: 80
- Finals: 80
- Project: 80

Expected Calculation Final Score = $(80 * 0.30) + (80 * 0.20) + (80 * 0.30) + (80 * 0.20)$

Final Score = 24.0 + 16.0 + 24.0 + 16.0

Final Score = 80.0

Expected Outcome

- Final Score: 80.0
 - Letter Grade: B
-

Scenario 3: Invalid Input Test Cases

Test Case 1:

Input

- Assignment: -10
- Midterm: 75
- Finals: 80
- Project: 85

Expected Calculation Input validation should fail for Assignment is less than 0, out of valid range.

Expected Outcome

- Final Score: Not Calculated
 - Letter Grade: Not Assigned
 - Error Message: Score must be between 0-100.
-

Test Case 2:

Input

- Assignment: 88
- Midterm: 92
- Finals: 105
- Project: 90

Expected Calculation Input validation should fail for Final Exam is greater than 100, out of valid range.

Expected Outcome

- Final Score: Not Calculated
- Letter Grade: Not Assigned
- Error Message: Score must be between 0-100.

8. Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full Name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
Primary Goal	Define detail system requirements before implementation	Describe how the system should behave from the user's perspective	Ensure code correctness through automated test
Main Focus	Requirements, rules, and specifications	User behavior, business process, and scenarios	Verification, reliability, and code quality
Key Question	What should the system do?	How should the system behave in real situations	Does the Implementation work properly?
Starting Point	System specifications & requirements documents	User stories & acceptance criteria	Test cases written before code
Typical Format	Functional specifications, requirements, diagrams	Given-When-Then scenarios	Unit tests & assertions
Primary Stakeholders	System analysts, architects, developers	Customers, product owners, testers, developers	Developers & QA Engineers
Development Process	Define specifications -> Design -> Implement	Define behavior scenarios -> Impement -> Validate	Write tests -> Write Code -> Refactor
Level of Detail	High-Level System requirements & constraints	User-oriented interactions & outcomes	Low-level code functionality
Advantages	Clear project scope &	Improves	Produces more reliable

Item	SDD	BDD	TDD
	requirements	communication between technical & non-technical members	& maintainable code
Potential Limitation	Specifications may become outdated if not maintained	Creating comprehensive behavior scenarios can be time-consuming	May require significant effort to write & maintain tests
Example for a Student Grade Calculator	Define grading rules, input ranges, output requirements	Describe scenarios such as calculating grades for passing or failing students	Create tests to verify grade calculations for different scores
Documentation Style	Requirement specifications and design documents	User stories & acceptance scenarios	Test suites & test reports
AI-Era Value	Helps AI understand what features & requirements must be implemented	Helps AI understand expected user interactions and system behavior	Helps verify if AI-generated code produces correct results
Success Measure	System meets document specifications	System behaves as expected in user scenarios	All test cases pass successfully
Best Used When	Planning & defining project requirements	Capturing users needs and business processes	Ensuring software quality & regression prevention

9. Reflection

After learning about SDD, BDD, and TDD, I found that TDD is the easiest approach for me to understand. As an IT student, I am already familiar with testing programs by providing inputs and checking whether the outputs are correct. TDD follows a similar mindset since it focuses on defining test cases before implementation. By thinking about expected results first, it becomes easier to identify errors and verify that a system behaves as intended.

When it comes to working with AI coding tools, I believe SDD is the most useful approach because it provides clear requirements and specifications. AI can generate code very quickly, but the quality of the generated result depends heavily on the instructions it receives. If requirements are vague, the AI may make assumptions that do not match the developer's intentions. A well-written SDD reduces ambiguity by clearly describing the system goals, inputs, outputs, and functional requirements.

SDD can help reduce unclear AI prompts by giving developers a structured way to define what the software should do. Instead of asking AI to "create a grade calculator," a developer can provide detailed requirements such as grading rules, calculation formulas, and validation constraints. This allows the AI to generate results that are closer to the expected solution.

BDD is useful because it describes software behavior from a user's perspective. The Given–When–

Then format makes requirements easier to understand and communicate. Rather than focusing on technical implementation details, BDD focuses on how users expect the system to behave in different situations. This can help AI tools understand the intended functionality and can also improve communication among developers, testers, and stakeholders.

TDD helps verify whether AI-generated code is correct by providing predefined test cases. Even if AI generates code that appears correct, it may still contain logical errors or fail in special situations. By running normal, boundary, and invalid-input test cases, developers can confirm whether the generated solution satisfies the requirements.

In future software dev. projects, I would combine all three approaches. I would start with SDD to define requirements and guide AI-generated outputs. Next, I would use BDD to describe user expectations and important system behaviors. Finally, I would apply TDD to create test cases and verify that the generated code works correctly. Using these three approaches together would provide a structured process for guiding, evaluating, and improving AI-assisted software development.

10. References & Usage of AI

References: [1] LinkedIn: <https://www.linkedin.com/pulse/specification-driven-development-sdd-true-parent-tdd-bdd-sampaio-ix7nf>

[2] Lunabase.AI: <https://lunabase.ai/blog/specification-driven-development-the-complete-guide-to-tdd-bdd-and-sdd-in-2025>

[3] testRigor: <https://testrigor.com/blog/tdd-vs-bdd-whats-the-difference-between-tdd-and-bdd/>

AI Tools: I mainly used ChatGPT & Microsoft Copilot for consulting on the requirements of this assignment, and giving me guidance & tips when I request it to, especially when I'm stuck and ran out of ideas.